

FaidaHai

The Backend, Database & Security

Presenter: Eman

Complete Study Guide & Presentation Preparation

Incubated at the Adan IT Center



FaidaHai — The Backend, Database & Security

Presenter: Eman | Role: Platform Architect

Complete Study Guide — Read This Fully Before the Presentation

PART 1: What is FaidaHai? (Start Here)

FaidaHai is a **website** (web application) that uses AI to help people make smarter real estate investment decisions in Pakistan. Users sign up, log in, search any neighborhood, and get instant AI-generated property valuations.

Your role in this presentation: Explain the "invisible engine" — the technology that makes everything work behind the scenes. The database, the security, the server, the login system.

PART 2: Understanding Web Development (From Zero)

How Does a Website Work?

When you open a website (like google.com), here's what happens:

You type "faidahai.com" in your browser



Your browser sends a REQUEST to a SERVER (a powerful computer somewhere in the world)



The server PROCESSES the request (finds the right page, fetches data)



The server sends back a RESPONSE (HTML, CSS, JavaScript files)



Your browser reads these files and DISPLAYS the website

The Two Halves of a Website

Every website has two parts:

Frontend (The Restaurant Dining Area)

- What users SEE and TOUCH
- Buttons, colors, text, images, animations
- Runs on the user's device (their laptop/phone browser)

Backend (The Kitchen)

- What happens BEHIND the scenes
- Saving data, running AI, checking passwords, securing information
- Runs on a SERVER (a computer in a data center somewhere)

You are presenting the kitchen. The user never sees the backend, but without it, nothing works.

PART 3: The Programming Languages We Used

What is a Programming Language?

A programming language is how humans tell computers what to do. Just like humans have English, Urdu, Arabic — computers have JavaScript, Python, SQL. Each language is good at different things.

Languages Used in FaidaHai

LANGUAGE	WHERE WE USE IT	WHAT IT DOES	REAL-WORLD ANALOGY
JavaScript	Everywhere — it's the foundation	The universal language of the web. Every website in the world uses it.	English — the global language everyone understands
TypeScript	Everywhere (we write TypeScript, which becomes JavaScript)	A SAFER version of JavaScript. It adds "rules" that catch mistakes before they cause problems.	English with a spell-checker and grammar checker built in
SQL	Database queries	The language used to talk to databases. "Show me all properties in Lahore" is written in SQL.	The filing system language — "find file #3427 from Cabinet B"

LANGUAGE	WHERE WE USE IT	WHAT IT DOES	REAL-WORLD ANALOGY
Python	Data pipeline (collecting property data)	Used for AI and data science tasks. We use it to collect and process property listings.	A Swiss Army knife — good at everything, especially data work
HTML	Web page structure	Defines what's ON the page (headings, paragraphs, buttons).	The blueprint of a building — where walls and doors go
CSS	Web page styling	Defines how things LOOK (colors, fonts, spacing).	The interior designer — paint colors, furniture style

Why TypeScript Instead of JavaScript?

Imagine you're writing a math test:

JavaScript: You can write `2 + "hello"` and the computer will say "2hello" — no error, just a weird result. This can cause hidden bugs.

TypeScript: You try to write `2 + "hello"` and BEFORE you even run the program, TypeScript says "ERROR: you can't add a number and text!" This catches bugs early.

For a commercial product like FaidaHai, TypeScript is essential because we can't afford bugs in production.

PART 4: The Tools & Frameworks

What is a "Framework"?

A framework is a pre-built structure that gives you a head start. Instead of building a house from raw bricks, a framework gives you the walls and you just decorate.

Our Technology Stack (The Complete Kitchen Equipment)

TOOL	CATEGORY	WHAT IT DOES	WHY WE CHOSE IT	WHO ELSE USES IT
Next.js 16	Web Framework	The main framework that runs our entire website. Handles pages, navigation, server code, and optimization.	It's the most powerful React framework. One codebase handles both frontend AND backend.	TikTok, Nike, Twitch, Hulu
Supabase	Database + Auth	Stores ALL our data and handles user login/signup.	It's like Firebase (Google's tool) but open-source, uses PostgreSQL (the most advanced database), and has built-in Row Level Security.	Used by startups and enterprises globally
PostgreSQL	Database Engine	The actual database engine inside Supabase. Stores data in tables (like Excel spreadsheets).	It's the world's most advanced open-source database. Banks and governments use it.	Instagram, Spotify, NASA
Vercel	Hosting/Deployment	Puts our website on the internet. When someone visits faidahai.vercel.app, Vercel serves the page.	Made by the same team that built Next.js. Gives us global CDN (fast everywhere in the world).	All Next.js applications
Google Gemini API	AI Service	The AI brain (Safoora covers this in detail). We send data to Google's servers and get AI analysis back.	Most powerful AI model available.	Developers worldwide
Node.js	Runtime Environment	Allows JavaScript to run on servers (not just browsers).	Essential for Next.js. JavaScript was originally only for browsers — Node.js freed it to run anywhere.	Netflix, LinkedIn, Uber

PART 5: The Database (The Most Important Part)

What is a Database?

A database is an organized collection of information stored on a computer. Think of it as a **digital filing cabinet** with perfectly organized folders.

In our case, the database stores:

- Every user who signs up
- Every AI search they've done
- Every property they've saved
- All the market data we've collected

What is PostgreSQL?

PostgreSQL (pronounced "post-gress-Q-L") is the **database engine** — the software that manages all this data. It's:

- **Free and open-source** (no license fees)
- **Used by Instagram, Spotify, and NASA**
- **Extremely reliable** — designed for mission-critical data
- **Supports vector operations** — this is crucial for our AI search

What is Supabase?

Supabase is a **service** that wraps around PostgreSQL and gives us:

1. **A dashboard** to see and manage our data visually
2. **Authentication** (login/signup system) built-in
3. **Row Level Security** (data protection)
4. **Real-time subscriptions** (data updates instantly)
5. **APIs** (ways for our code to talk to the database)

Think of PostgreSQL as the engine, and Supabase as the entire car around it.

Our Database Tables

A "table" is like an Excel spreadsheet. Each table stores one type of data.

Table 1: `auth.users` (Managed by Supabase)

COLUMN	WHAT IT STORES	EXAMPLE
id	Unique user identifier	<code>a1b2c3d4-e5f6-...</code>
email	User's email address	<code>areeba@gmail.com</code>
encrypted_password	Password (encrypted, not readable)	<code>\$2b\$10\$xK9z...</code>
created_at	When they signed up	<code>2026-05-15 14:30:00</code>

Table 2: `property_analysis` (AI Search History)

COLUMN	WHAT IT STORES	EXAMPLE
id	Unique analysis ID	<code>f7g8h9i0-...</code>
user_id	Which user did this search	Links to <code>auth.users</code>
city	What neighborhood they searched	<code>DHA Phase 9</code>
floors	Number of floors	<code>3</code>
area	Area in sqft	<code>2000</code>
estimated_value	AI's estimated value	<code>42000000</code> (4.2 Crore)
projected_roi	Projected return	<code>14.5</code> (percent)
key_strengths	Positive factors	<code>["Prime location", "High demand"]</code>
risk_factors	Risks identified	<code>["Market oversupply"]</code>
created_at	When this analysis was done	<code>2026-05-18 20:15:00</code>

Table 3: `user_portfolios` (Saved Properties)

COLUMN	WHAT IT STORES	EXAMPLE
id	Unique portfolio entry ID	<code>j1k2l3m4-...</code>

COLUMN	WHAT IT STORES	EXAMPLE
user_id	Which user owns this	Links to auth.users
property_name	Name the user gave it	My DHA Plot
city	City/location	Lahore
estimated_value	Current estimated value	35000000 (3.5 Crore)
projected_roi	Expected annual return	12.5

Table 4: `market_data` (Real Estate Listings)

COLUMN	WHAT IT STORES	EXAMPLE
id	Unique listing ID	n5o6p7q8-...
title	Property title	Luxury 5 Marla House
location	Where it is	DHA Phase 8, Lahore
price	Listed price	32000000
property_type	Type	Residential
area_sqft	Size	1125
embedding	AI numbers (768 dimensions)	[0.23, -0.45, ...]

PART 6: Security — Row Level Security (RLS)

The Problem

Imagine 1,000 users are saving their portfolios to the SAME database. How do we make sure User A can't see User B's private investment data?

Bad Solution (What Beginners Do)

Check permissions in the application code:

```
if (current_user.id === data.user_id) {  
  show the data  
} else {  
  deny access  
}
```

Why this is bad: If a hacker bypasses the application code and talks to the database directly, they can see everything. The security is only at the "door" — once past it, there's no protection.

Good Solution (What We Do): Row Level Security

RLS puts the security IN THE DATABASE ITSELF. Even if a hacker bypasses our application, the database STILL denies access.

Analogy:

Imagine a bank vault with 1,000 safe deposit boxes. Each box can only be opened with the owner's fingerprint. Even if someone breaks into the vault room, they STILL can't open any box that isn't theirs. The security is on EACH BOX, not just on the vault door.

That's RLS. Each ROW (piece of data) in the database has a rule: "Only the user who created this row can see/edit/delete it."

Our Actual Security Rules (in Simple Terms)

RULE 1: "Users can only INSERT data tagged with their own user ID"

→ You can only add your OWN data, not pretend to be someone else

RULE 2: "Users can only SELECT (view) data tagged with their own user ID"

→ You can only see YOUR OWN searches and portfolio

RULE 3: "Users can only UPDATE their own data"

→ You can only edit YOUR OWN entries

RULE 4: "Users can only DELETE their own data"

→ You can only remove YOUR OWN entries

This is bank-level security. This is enterprise-grade. This is what separates FaidaHai from hobby projects.

PART 7: The Login System (Authentication)

How Does Login/Signup Work?

USER SIGNS UP:

1. User enters email + password
2. Supabase creates account in auth.users table
3. Password is ENCRYPTED (hashed) – even WE can't read it
4. Supabase sends a confirmation email
5. User confirms → Account is active

USER LOGS IN:

1. User enters email + password
2. Supabase checks if the email exists
3. Supabase compares the encrypted password
4. If match → Creates a SESSION TOKEN (like a digital wristband)
5. This token is stored in the browser as a "cookie"
6. Every page load, the token is checked automatically
7. If token is valid → User stays logged in
8. If token expires or is invalid → User is redirected to login

USER LOGS OUT:

1. User clicks "Logout"
2. The session token is DESTROYED
3. User is sent to the login page
4. They can't access the dashboard until they log in again

What is a Session Token?

Think of it like a **wristband at a concert**:

- When you buy a ticket (login), you get a wristband (token)
- Every area you try to enter, security checks your wristband
- If your wristband is valid, you get in
- When the concert ends (logout), the wristband is cut off
- You can't get back in without a new ticket (logging in again)

What is Password Hashing?

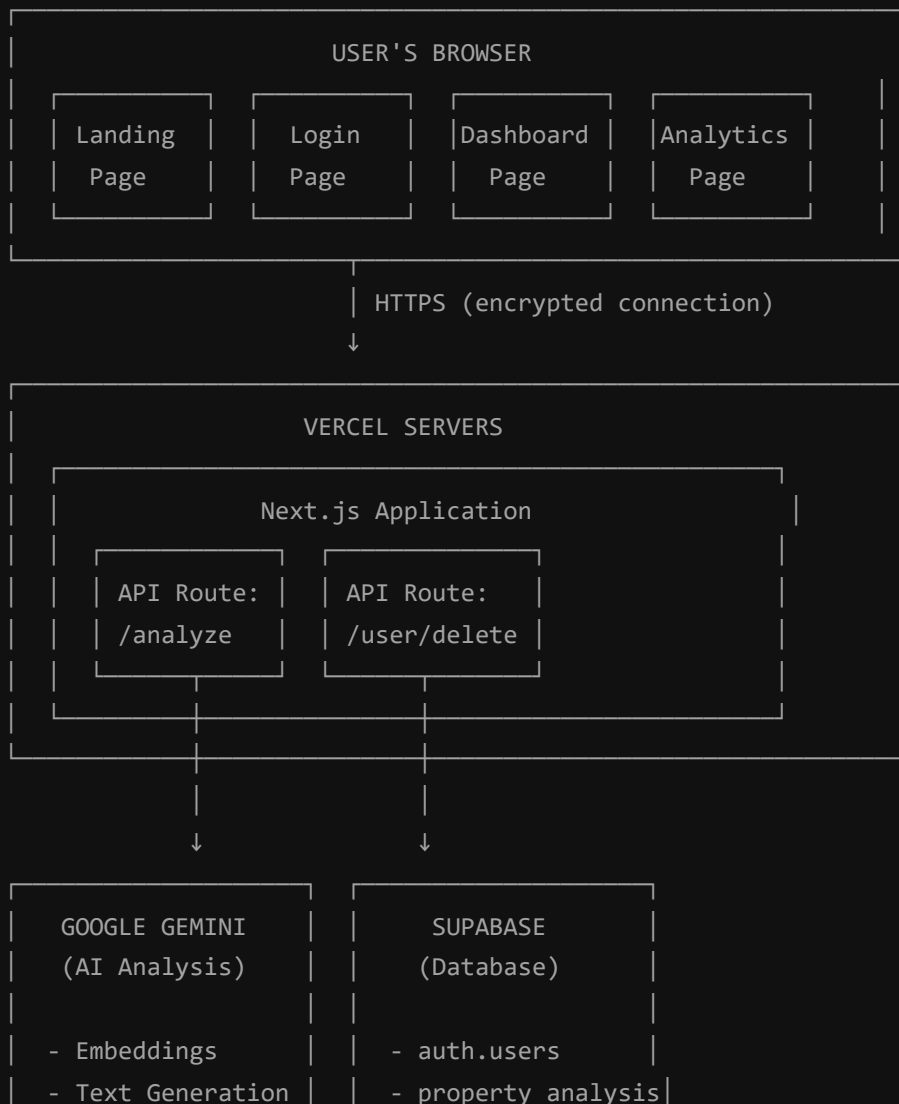
We NEVER store passwords as plain text. Instead:

- Password: `myPassword123`
- Stored as: `$2b$10$xK9zQp3R1m8Jm5kZ7...` (unreadable)

This is called "hashing" — it's a one-way process. You can convert a password TO a hash, but you can NEVER convert the hash BACK to the password. Even our database administrators can't see user passwords.

PART 8: The Data Flow (How Everything Connects)

Complete System Architecture



- user_portfolios
- market_data

PART 9: Account Deletion (Secure Backend Process)

Why This Matters

For a commercial platform, users must be able to permanently delete their accounts. This is required by privacy regulations and builds user trust.

How We Implemented It (Securely)

USER clicks "Delete My Account"

↓

A CONFIRMATION modal appears – they must type "DELETE"

↓

Frontend sends a DELETE request to `/api/user/delete`

↓

The SERVER first verifies the user is really logged in

↓

The server uses a SPECIAL KEY (Service Role Key) that has admin-level access to the database

↓

It deletes ALL of the user's data:

1. All `property_analysis` rows
2. All `user_portfolios` rows
3. The `auth.users` record itself

↓

The user is logged out and redirected to the homepage

Why Server-Side?

Important security point: We do NOT let the browser (client-side) delete data directly. Why?

If deletion was client-side:

- A hacker could modify the code in the browser

- They could potentially delete OTHER users' data
- The admin key would be exposed in the browser (CATASTROPHIC)

By doing it server-side:

- The admin key is ONLY on the server (never visible to users)
- The server first verifies the user's identity
- Only the authenticated user's data is deleted
- No one can abuse this to delete others' data

PART 10: What is an API? (Frequently Asked)

Simple Explanation

API stands for **Application Programming Interface**. It's a way for two computer programs to talk to each other.

Analogy:

Imagine you're at a restaurant. You (the frontend) want food (data). The kitchen (the backend/database) has the food. But you can't go into the kitchen yourself. Instead, you tell the WAITER (the API) what you want, and the waiter brings it to you.

An API is the waiter between the frontend and the backend.

Our API Routes

ROUTE	METHOD	WHAT IT DOES
<code>/api/analyze</code>	POST	Receives a property query, runs AI analysis, returns results
<code>/api/user/delete</code>	DELETE	Permanently deletes user account and all data
<code>/api/heatmap</code>	GET	Returns market data for the heatmap visualization

PART 11: What is Deployment? (How the Website Goes Live)

The Journey from Code to Live Website

Step 1: We write code on our LAPTOP (local development)

↓

Step 2: We test it locally (npm run dev – opens on localhost:3000)

↓

Step 3: We PUSH the code to GitHub (an online code storage platform)

↓

Step 4: Vercel DETECTS the new code on GitHub automatically

↓

Step 5: Vercel BUILDS the website (compiles TypeScript, optimizes images, etc.)

↓

Step 6: Vercel DEPLOYS it to their global CDN (servers around the world)

↓

Step 7: Users can access it at faidahai.vercel.app

What is a CDN?

CDN = Content Delivery Network. It's a network of servers spread across the world.

Without CDN: A user in Lahore requests the website → request goes to a server in USA → slow response.

With CDN: A user in Lahore requests the website → request goes to the nearest server (maybe Singapore or Mumbai) → fast response.

Vercel has servers in **100+ locations worldwide**.

PART 12: Complete Q&A Preparation

Q: "What programming language did you use?"

"The entire platform is written in TypeScript, which is a safer, more reliable version of JavaScript — the universal language of the web. For our data collection pipeline, we used Python."

Q: "Where is the data stored?"

"In a PostgreSQL database hosted on Supabase. PostgreSQL is the world's most advanced open-source database — it's the same technology used by Instagram and Spotify."

Q: "Is user data safe?"

"Absolutely. We implement Row Level Security at the database level, which means even if someone bypasses our application code, the database itself enforces that each user can only see their own data. Passwords are hashed using bcrypt — even our administrators can't see them."

Q: "How is the website hosted?"

"On Vercel's global CDN with servers in over 100 locations worldwide. This ensures fast load times regardless of where the user is accessing from."

Q: "Can this handle many users at once?"

"Yes. Vercel auto-scales — meaning if 10 people or 10,000 people visit at the same time, Vercel automatically allocates more server resources. Supabase also scales its database connections automatically."

Q: "What happens if the server goes down?"

"Vercel guarantees 99.99% uptime. Even if one server fails, the CDN routes traffic to another server automatically. For the database, Supabase provides daily backups."

Q: "What is Next.js?"

"Next.js is a web framework built on top of React — the most popular frontend library in the world, created by Facebook. Next.js adds server-side capabilities, routing, and performance optimization on top of React. Companies like TikTok and Nike use it."

Q: "How do you protect against hackers?"

"Multiple layers: HTTPS encryption for all data in transit, password hashing with bcrypt, Row Level Security in the database, server-side API routes that never expose secret keys, and session-based authentication with automatic token expiration."

PART 13: Key Terms Glossary

TERM	SIMPLE DEFINITION
Backend	The server-side code that users never see (database, AI processing, security)
Frontend	The client-side code users interact with (buttons, pages, animations)
Database	Organized digital storage for all application data
PostgreSQL	The world's most advanced open-source database engine
Supabase	A platform that provides database + authentication + security
API	A "waiter" that carries requests between frontend and backend
Server	A powerful computer that processes requests and serves web pages
TypeScript	A safer version of JavaScript with built-in error checking
RLS	Row Level Security — database-level protection per data row
Hashing	Converting passwords to unreadable strings (one-way, irreversible)

TERM	SIMPLE DEFINITION
Session Token	A digital "wristband" that proves you're logged in
CDN	Content Delivery Network — servers around the world for fast loading
Deployment	The process of putting code onto the internet for public access
HTTPS	Encrypted connection between browser and server
Environment Variable	Secret values (API keys, passwords) stored on the server, never in code
Service Role Key	A special admin key that can bypass RLS (only on the server)

PART 14: Numbers & Facts to Memorize

- 🗄️ **Database:** PostgreSQL via Supabase
- 🌐 **Hosting:** Vercel (100+ global CDN locations)
- 🛡️ **Security:** Row Level Security + bcrypt hashing
- 🧑💻 **Language:** TypeScript (safer JavaScript)
- 📊 **Core Tables:** 4 (users, analysis, portfolios, market_data)
- 🗑️ **Deletion:** Secure server-side with admin Service Role Key
- 📡 **API Routes:** 3 (`/analyze` , `/user/delete` , `/heatmap`)
- ⌚ **Uptime Guarantee:** 99.99% (Vercel)
- 🔑 **Password Storage:** bcrypt hashed (irreversible)
- 🛠️ **Framework:** Next.js 16 (used by TikTok, Nike)

made with love by HAMZA